

# Proyecto DevOps - NodeJs

## CI/CD, Calidad de Código e Infraestructura para una aplicación NestJS

### 1. Contexto

Como parte de la formación de la ruta **Node.js** → **Express** → **NestJS**, se desarrollará un proyecto orientado a introducir al coder en prácticas fundamentales de **DevOps**, **integración continua**, **entrega continua**, **automatización**, **calidad de software** e **infraestructura**.

El objetivo es construir, de manera incremental, un esquema que permita llevar una aplicación desarrollada en **NestJS** desde el repositorio de código fuente hasta un ambiente de ejecución, aplicando controles automáticos durante el proceso.

La solución será implementada sobre un laboratorio virtualizado utilizando **Proxmox**, donde cada componente de la plataforma tendrá una responsabilidad específica.

El proyecto no busca formar especialistas en infraestructura, Kubernetes o seguridad avanzada. Busca que el desarrollador comprenda y sea capaz de participar en un ciclo básico y profesional de desarrollo y despliegue.

### 2. Objetivo general

Diseñar e implementar un flujo DevOps básico para una aplicación NestJS que permita:

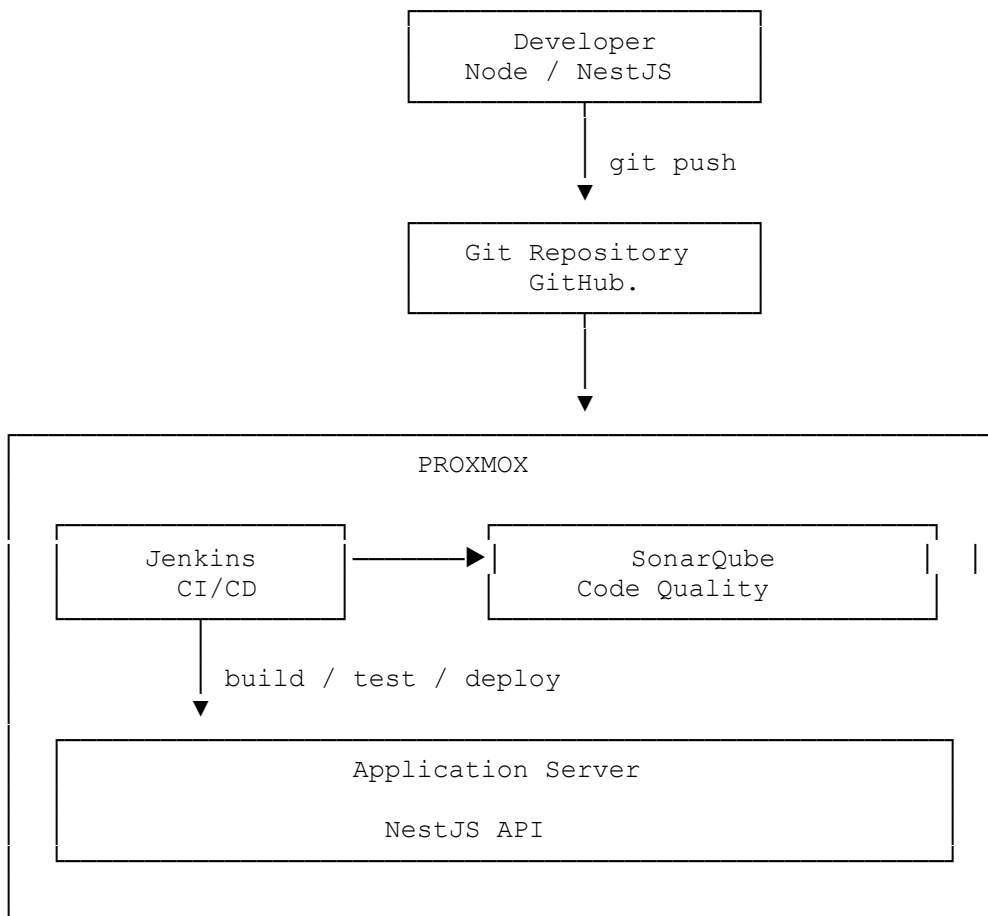
- Gestionar el código mediante Git.
- Aplicar convenciones de commits.
- Ejecutar validaciones automáticas antes de realizar commits.
- Integrar análisis de calidad mediante SonarQube.
- Ejecutar pruebas automatizadas.
- Medir cobertura de pruebas.
- Detectar problemas de calidad, duplicación y code smells.
- Implementar un pipeline CI/CD mediante Jenkins.
- Construir la aplicación automáticamente.
- Desplegar la aplicación en un servidor Linux.
- Actualizar automáticamente la aplicación cuando existan cambios aprobados en la rama `master`.

- Utilizar variables de entorno para la configuración.
- Aplicar controles básicos de seguridad.
- Comprender la separación de responsabilidades entre desarrollo, integración, calidad y despliegue.

### 3. Arquitectura propuesta

El laboratorio será construido sobre **Proxmox**.

Cada máquina virtual tendrá una responsabilidad específica.



## 4. Infraestructura del laboratorio

Se propone inicialmente utilizar cuatro máquinas virtuales.

| VM   | Responsabilidad            | Tecnología                |
|------|----------------------------|---------------------------|
| VM01 | Jenkins                    | Ubuntu Server + Jenkins   |
| VM02 | SonarQube                  | Ubuntu Server + SonarQube |
| VM03 | Aplicación                 | Ubuntu Server + Node.js   |
| VM04 | Git / servicios auxiliares | Git externo               |

No es obligatorio que Git esté dentro de Proxmox. Para reducir la complejidad del laboratorio, se puede utilizar **GitHub** como repositorio remoto.

En ese escenario:

```
Proxmox
├── VM01 Jenkins
├── VM02 SonarQube
└── VM03 Application Server
```

## 5. Responsabilidad de cada componente

### Git

Responsable de:

- Control de versiones.
- Branches.
- Pull/Merge Requests.
- Historial de cambios.
- Identificación de historias de usuario.
- Integración con Jenkins.

## Git Hooks

Responsables de realizar validaciones locales antes de permitir determinados eventos Git.

Ejemplos:

- Formato del mensaje del commit.
- Ejecución de lint.
- Ejecución de pruebas.
- Validación de archivos.
- Prevención de commits que no cumplan las reglas.
- Ejecutar el SonarQube

## SonarQube

Responsable del análisis estático del código.

Se comprenderán conceptos como:

- Bugs.
- Vulnerabilidades.
- Code Smells.
- Código duplicado.
- Cobertura de pruebas.
- Maintainability.
- Reliability.
- Security.
- Quality Gate.

El objetivo es comprender que **compilar correctamente no significa necesariamente tener código de calidad.**



## Jenkins

Responsable de automatizar el proceso CI/CD.

Jenkins deberá:

1. Detectar cambios.
2. Obtener el código.
3. Instalar dependencias.
4. Construir la aplicación.
5. Desplegarla.

## Application Server

Responsable exclusivamente de ejecutar la aplicación NestJS.

El servidor no deberá utilizarse para desarrollar.

Su responsabilidad será:

```
Código desplegado
↓
Node.js
↓
NestJS
↓
API
```

## 6. Estrategia de ramas

Se propone utilizar una estrategia sencilla:

```
master
├── feature/US-101-login
├── feature/US-102-customers
└── feature/US-103-products
```

Las funcionalidades se desarrollan en ramas independientes.

Ejemplo:

```
feature/US-101-login
```

Cuando la funcionalidad está terminada:

```
feature/US-101-login
    ↓
Pull Request
    ↓
master
```

La rama `master` será considerada la rama desplegable.



## 7. Convención de commits

Cada commit deberá estar relacionado con una historia de usuario.

Formato propuesto:

```
[US-XXX] tipo: descripción
```

Ejemplos:

```
[US-101] feat: agregar autenticación de usuarios
```

```
[US-102] fix: corregir validación de clientes
```

```
[US-103] test: agregar pruebas para customers service
```

```
[US-104] refactor: simplificar customer repository
```

Tipos permitidos:

```
feat  
fix  
test  
refactor  
docs  
chore
```

Ejemplo inválido:

```
arreglo login
```

Ejemplo válido:

```
[US-125] fix: corregir validación de contraseña
```

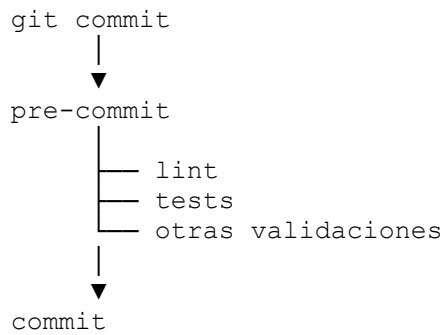
## 8. Validaciones locales

Antes de permitir un commit, se deberán ejecutar validaciones mediante Git Hooks.

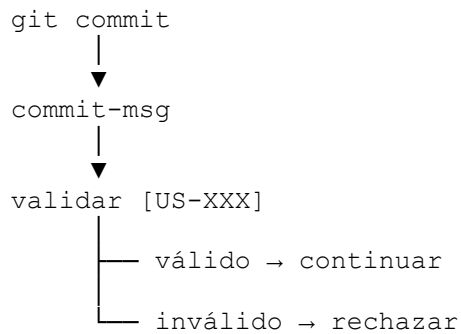
Se propone utilizar una herramienta como:

Husky

El flujo conceptual será:



Para el mensaje:

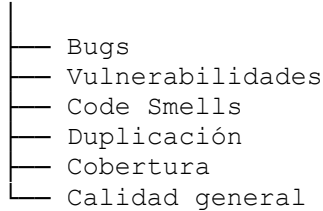


## 9. Integración con SonarQube

El proyecto NestJS deberá ser analizado por SonarQube.

SonarQube analizará, entre otros:

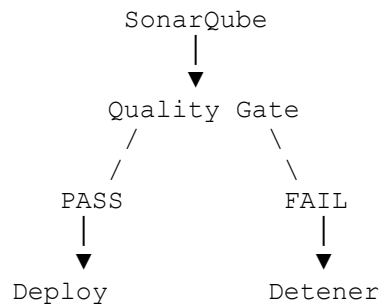
Código fuente



## 10. Quality Gate

El pipeline no deberá continuar automáticamente hacia el despliegue si el análisis de calidad no cumple las condiciones definidas.

Conceptualmente:



Los criterios deberán ser definidos previamente.

Por ejemplo:

- Cobertura mínima.
- Sin nuevos bugs críticos.
- Sin nuevas vulnerabilidades críticas.
- Límite de duplicación.
- Code smells dentro del umbral definido.

Los valores exactos serán definidos como parte del ejercicio y deberán justificarse técnicamente.

Para esto se debe utilizar la API de SonarQube de tal forma que se pueda consultar la información necesaria para continuar con el punto 11.

## 11. Pipeline CI/CD

El pipeline deberá implementar como mínimo las siguientes etapas:

```
Checkout
↓
Install dependencies
↓
Lint
↓
Unit Tests
↓
Coverage
↓
SonarQube Analysis
↓
Quality Gate
↓
Build
↓
Deploy
↓
Health Check
```

## 12. Ejemplo conceptual de Jenkinsfile

El proyecto deberá terminar con un pipeline versionado dentro del repositorio.

Ejemplo conceptual:

```
pipeline {
    stages {
        stage('Checkout') {
            ...
        }

        stage('Install') {
            ...
        }

        stage('Lint') {
            ...
        }
    }
}
```



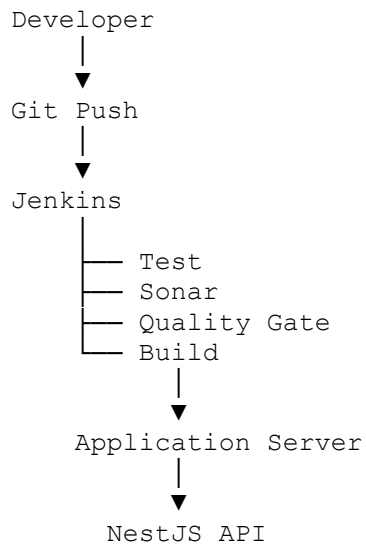
```
    stage('Test') {  
        ...  
    }  
  
    stage('Coverage') {  
        ...  
    }  
  
    stage('SonarQube') {  
        ...  
    }  
  
    stage('Quality Gate') {  
        ...  
    }  
  
    stage('Build') {  
        ...  
    }  
  
    stage('Deploy') {  
        ...  
    }  
  
    stage('Health Check') {  
        ...  
    }  
}  
}
```

El objetivo del ejercicio no será memorizar la sintaxis de Jenkins, sino comprender qué responsabilidad tiene cada etapa.

## 13. Despliegue

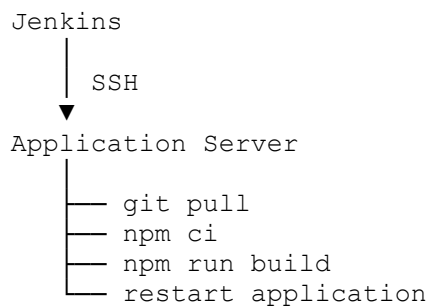
Una vez aprobado el Quality Gate, Jenkins deberá desplegar la aplicación.

Flujo:



El despliegue puede realizarse inicialmente mediante SSH.

Por ejemplo:



## 14. Variables de entorno

La aplicación no deberá contener información sensible directamente en el código.

Ejemplos:

```
NODE_ENV=production
PORT=3000
```

```
DB_HOST=...
DB_PORT=...
DB_NAME=...
DB_USER=...
DB_PASSWORD=...
```

El ejercicio deberá introducir conceptos básicos de:

- .env
- .env.example
- Variables de entorno.
- Secrets.
- Credenciales de Jenkins.
- Separación entre configuración y código.

## 15. Seguridad básica

El laboratorio deberá incluir controles básicos.

### Git

- No almacenar passwords.
- No almacenar tokens.
- No almacenar archivos .env.
- Utilizar .gitignore.
- Revisar secretos antes de realizar commits.

### Jenkins

- No almacenar credenciales directamente en el Jenkinsfile.
- Utilizar Jenkins Credentials.
- Utilizar usuarios con permisos mínimos necesarios.

## Servidor

- Usuario de despliegue independiente.
- Evitar trabajar directamente como `root`.
- SSH mediante llave.

## Aplicación

- Variables de entorno.
- Validación de entrada.
- Manejo adecuado de errores.
- No exponer información sensible en logs.

# 16. Health Check

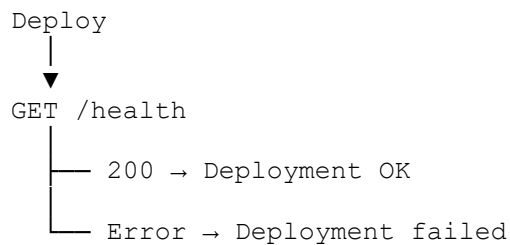
La aplicación deberá disponer de un endpoint sencillo:

```
GET /health
```

Respuesta esperada:

```
{  
  "status": "ok"  
}
```

Jenkins utilizará este endpoint después del despliegue.



Esto permite introducir un concepto importante:

**desplegar no significa necesariamente que la aplicación esté funcionando.**

## 17. Plan de trabajo

El proyecto se desarrollará durante una semana.

### Día 1 — Introducción a DevOps e infraestructura

#### Objetivos

Comprender:

- Qué es DevOps.
- CI.
- CD.
- CI/CD.
- Git Flow básico.
- Infraestructura.
- Servidores.
- Virtualización.
- Proxmox.
- Separación de responsabilidades.

#### Actividad

Construir la infraestructura inicial:

```
Proxmox
├── Jenkins VM
├── SonarQube VM
└── Application VM
```

Cada coder deberá comprender qué función cumple cada máquina.

#### Entregable

Laboratorio funcionando y documentación de arquitectura por células.



## Día 2 — Git, branches y Git Hooks

### Objetivos

Trabajar:

- Branches.
- Pull Requests.
- Convenciones de commits.
- Git Hooks.
- Husky.
- Lint.
- Pre-commit.
- Commit-msg.

### Actividad

Implementar:

```
feature/US-XXX
```

y validar:

```
[US-XXX] tipo: descripción
```

antes de aceptar el commit.

### Entregable

Repositorio con:

```
.husky/  
package.json  
.gitignore  
README.md
```

y hooks funcionales.

Recuerda

**ESLint pregunta:** "¿Este código cumple nuestras reglas?"

**Jest pregunta:** "¿Este código funciona según nuestras pruebas?"

**SonarQube pregunta:** "¿Qué tan saludable es este código?"

**Jenkins pregunta:** "¿Podemos automatizar todo este proceso?"

# Día 3 — Calidad de código y SonarQube

## Objetivos

Comprender:

- Static Code Analysis.
- Bugs.
- Vulnerabilidades.
- Code Smells.
- Duplicación.
- Cobertura.
- Quality Gate.

## Actividad

Conectar el proyecto NestJS con SonarQube.

Ejecutar:

```
lint
tests
coverage
sonar analysis
```

Configurar el Quality Gate.

## Entregable

Dashboard SonarQube funcionando y análisis asociado al proyecto.

## Día 4 — Jenkins y CI

### Objetivos

Comprender:

- Jenkins.
- Jobs.
- Pipeline.
- Webhooks.
- Jenkinsfile.
- CI.

### Actividad

Crear el pipeline:

```
Checkout
↓
Install
↓
Lint
↓
Test
↓
Coverage
↓
SonarQube
↓
Quality Gate
```

El pipeline deberá detenerse cuando alguna validación obligatoria falle.

### Entregable

Pipeline CI ejecutándose automáticamente.

# Día 5 — CD y despliegue

## Objetivos

Comprender:

- Continuous Delivery.
- Continuous Deployment.
- Deployment.
- SSH.
- Variables de entorno.
- Secrets.
- Health Checks.

## Actividad

Completar:

```
Git
↓
Jenkins
↓
Tests
↓
SonarQube
↓
Quality Gate
↓
Build
↓
Deploy
↓
Health Check
```

El servidor deberá terminar ejecutando la aplicación NestJS.

## Entregable

Aplicación desplegada automáticamente.

## 18. Reto final

Al finalizar la semana, cada equipo deberá realizar una modificación sobre la aplicación NestJS.

Ejemplo:

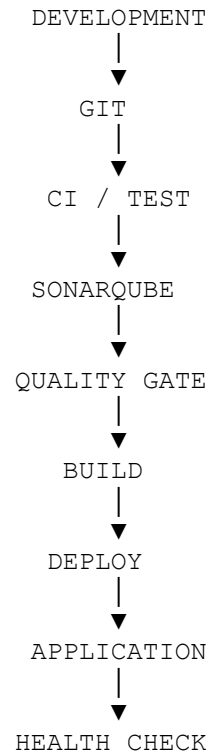
Historia:

US-200 - Crear endpoint para consultar productos

El equipo deberá:

1. Crear branch.
2. Implementar funcionalidad.
3. Crear pruebas.
4. Ejecutar validaciones locales.
5. Realizar commit utilizando la convención.
6. Hacer push.
7. Generar Pull Request.
8. Integrar los cambios a `master`.
9. Activar Jenkins.
10. Ejecutar pruebas.
11. Ejecutar SonarQube.
12. Validar Quality Gate.
13. Construir la aplicación.
14. Desplegar.
15. Ejecutar Health Check.
16. Verificar la aplicación desplegada.

El objetivo es completar el ciclo completo:



## 19. Entregables finales

Al finalizar el proyecto deberán existir:

### Infraestructura

- Laboratorio Proxmox.
- VM Jenkins.
- VM SonarQube.
- VM Application Server.
- Red configurada.
- Accesos documentados.

### Código

- Aplicación NestJS.
- Git.
- Branches.
- Git Hooks.
- Convención de commits.
- Tests.



- Coverage.
- Variables de entorno.

## **DevOps**

- Jenkinsfile.
- Pipeline CI/CD.
- Integración SonarQube.
- Quality Gate.
- Deployment automático.
- Health Check.

## **Documentación**

El equipo deberá entregar un documento que explique:

1. Arquitectura.
2. Infraestructura.
3. Flujo Git.
4. Estrategia de branches.
5. Convención de commits.
6. Git Hooks.
7. SonarQube.
8. Quality Gate.
9. Jenkins.
10. Pipeline.
11. Deployment.
12. Variables de entorno.
13. Seguridad.
14. Evidencias del despliegue.

## 20. Criterio fundamental del proyecto

El objetivo no es que el coder aprenda a administrar servidores como un administrador de infraestructura.

El objetivo es que un desarrollador Node/NestJS sea capaz de entender y participar en el ciclo completo:

**"Escribo código → lo valido → lo pruebo → mido su calidad → lo integro → lo construyo → lo despliego → verifico que funciona."**

De esta manera, el desarrollo deja de verse como una actividad aislada y pasa a formar parte de un proceso automatizado de entrega de software.